Quicksort solutions

07-E01 Lomuto partition trace

For j=1..5 the comparisons are 7>2,3>2,8>2,3>2,6>2, so none causes a swap. The pivot itself at j=6 is not scanned. Finally exchange positions 1 and 6 to obtain [2,3,8,3,6,7], with q=1. The left side is empty; the right side is [3,8,3,6,7]. This illustrates the most unbalanced split from a minimum pivot. A near-median pivot on the same input could split better.

07-E02 A 1:9 recursion bound

Total problem size on every level is at most n. Along the longest path sizes shrink by factor .9, so O(log n) levels reach size 1. All other paths stop no later; total cost is at most a constant times n log n. The argument applies whenever the larger child is at most .9 times its parent; it does not require every split to be exactly 1:9.

07-E03 Comparing four-rank endpoints

Among [1,2,3,4], the first pivot is uniform. Endpoints compare iff the first pivot is 1 or 4, giving 2/4=1/2. A first pivot 2 places 1 left and 4 right forever; pivot 3 does the same. An interior first pivot separates the endpoints, explaining why the probability is not 1.

07-E04 Equal keys and random pivots

Every selected pivot has value 5, every $\leq$ test succeeds, and q=r. Thus $C(n) = C(n-1) + n - 1$,C(0)=C(1)=0, giving n(n−1)/2. Random choices never change child sizes, so expectation equals this fixed value.

This two-way expected O(n log n) argument needs distinct keys. Three-way partition gives <5,=5,>5. Here all n keys belong to the equal region; both strict sides are empty and no further recursion occurs, giving Θ(n). In general recurse on only the two strict regions, never the equal region again.

07-E05 Smaller-side recursion stack

Child sizes satisfy a+b=n−1, so min(a,b)≤floor((n−1)/2). Actual recursive frame sizes at least halve, bounding depth by O(log(n+1)). The while loop adds no frame for the larger side.

When every pivot is maximum, the smaller side is empty and the while loop continues on n−1. Stack can even stay constant, yet partition comparisons sum (n−1)+(n−2)+…+1=Θ(n^2). Reducing frames does not eliminate partition work; space and time are separate resources.

Quicksort solutions (continued)

07-E06 Negative and repeated keys

Initially i=0. At j=1, −1≤2 gives i=1 and a self-swap, leaving the array unchanged. At j=2,4>2 leaves i=1. At j=3,0≤2 gives i=2 and swap 2,3, producing [−1,0,4,4,2]. At j=4,4>2 leaves i=2. Finally swap i+1=3 with r=5, producing [−1,0,2,4,4],q=3.

Left [−1,0] is ≤2 and right [4,4] is >2. They happen to be sorted here, but partition guarantees only cross-region relations. The teaching example yields left [3,1,2] and right [8,7,9], each still requiring recursive sorting.

Verification code appendix: partition_trace.py

```
"""Trace Lomuto partition on the course example."""

def partition(a):
    pivot = a[-1]
    i = -1
    for j in range(len(a) - 1):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[-1] = a[-1], a[i + 1]
    return i + 1


sample = [7, 3, 8, 3, 6, 2]
pivot_index = partition(sample)
assert all(x <= sample[pivot_index] for x in sample[:pivot_index])
assert all(x > sample[pivot_index] for x in sample[pivot_index + 1:])
print(sample, pivot_index)
```

Lin Cing
github.com/xinhi6716/computer-science-self-study

Quicksort solutions (continued)

Verification code appendix: quicksort_checks.py

```
"""Trace Lomuto comparisons and check exact finite expectations independently."""
from fractions import Fraction
from itertools import permutations,product
from math import factorial,log2,ceil


def sort(values,small_stack=False):
    a=list(values)
    comparisons=[]
    deepest=0
    def partition(p,r):
        pivot=a[r]
        i=p-1
        for j in range(p,r):
            comparisons.append((a[j],pivot))
            if a[j]<=pivot:
                i+=1
                a[i],a[j]=a[j],a[i]
        a[i+1],a[r]=a[r],a[i+1]
        return i+1
    def run(p,r,depth=1):
        nonlocal deepest
        deepest=max(deepest,depth)
        while p<r:
            q=partition(p,r)
            if not small_stack:
                run(p,q-1,depth+1)
                run(q+1,r,depth+1)
                return
            if q-p<r-q:
                run(p,q-1,depth+1)
                p=q+1
            else:
                run(q+1,r,depth+1)
                r=q-1
    run(0,len(a)-1)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quicksort solutions (continued)

```
    return a,comparisons,deepest


def main():
    cases=0
    for length in range(8):
        for values in product(range(3),repeat=length):
            for reduced_stack in (False,True):
                result,_,depth=sort(values,reduced_stack)
                assert result==sorted(values)
                if reduced_stack:
                    assert depth<=ceil(log2(length+1))+1
            cases+=1
    for n in range(2,8):
        total=0
        pair_counts={}
        for values in permutations(range(1,n+1)):
            _,comp,_=sort(values)
            total+=len(comp)
            for x,y in comp:
                pair=tuple(sorted((x,y)))
                pair_counts[pair]=pair_counts.get(pair,0)+1
        h=sum((Fraction(1,k) for k in range(1,n+1)),Fraction())
        assert Fraction(total,factorial(n))==2*(n+1)*h-4*n
        for (i,j),count in pair_counts.items():
            assert Fraction(count,factorial(n))==Fraction(2,j-i+1)
    for n in (4,10,64):
        _,comparisons,depth=sort([5]*n,True)
        assert len(comparisons)==n*(n-1)//2 and depth<=2
    print(f"PASS quicksort_arrays={cases} exact_permutation_means_n2_to7=True
pair_probabilities=True all_equal_quadratic=True smaller_side_stack=True")


if __name__=="__main__":
    main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study